



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C15

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji, **Interfejsy i klasy abstrakcyjne**, Przeciążanie operatorów, Wzorce projektowe (Design Patterns), Kompozycja vs. dziedziczenie, Zasada pojedynczej odpowiedzialności (Single Responsibility Principle), Zasada otwarte-zamknięte (Open-Closed Principle), Zasada podstawienia Liskov (Liskov Substitution Principle), Zasada segregacji interfejsów (Interface Segregation Principle), Zasada odwrócenia zależności (Dependency Inversion Principle), Testowanie jednostkowe klas i metod.

Interfejsy i klasy abstrakcyjne w C#

W programowaniu obiektowym interfejsy i klasy abstrakcyjne są narzędziami, które pozwalają na definiowanie wspólnych zachowań i struktur dla klas pochodnych. Mają one na celu zwiększenie elastyczności, modularności i czytelności kodu.

1. Interfejsy

Interfejs to kontrakt, który definiuje zestaw metod, właściwości lub zdarzeń, które muszą być zaimplementowane przez klasy go implementujące. Interfejsy nie zawierają implementacji metod (z wyjątkiem domyślnych metod w C# 8.0+). Klasa może implementować wiele interfejsów, co pozwala na wielokrotne dziedziczenie zachowań.

2. Klasy abstrakcyjne:

Klasa abstrakcyjna to klasa, która nie może być instancjonowana (nie można tworzyć jej obiektów). Może zawierać zarówno metody z implementacją, jak i metody abstrakcyjne (bez implementacji), które muszą być zaimplementowane przez klasy pochodne. Klasa może dziedziczyć tylko po jednej klasie abstrakcyjnej.

Przykład 1. Interfejsy

```
using System;  
  
// Definicja interfejsu  
public interface ILatajacy
```

```

{
    void Lataj(); // Metoda bez implementacji
}

// Klasa implementująca interfejs
public class Ptak : ILatajacy
{
    public void Lataj()
    {
        Console.WriteLine("Ptak lata machając skrzydłami.");
    }
}

// Druga klasa implementująca ten sam interfejs
public class Samolot : ILatajacy
{
    public void Lataj()
    {
        Console.WriteLine("Samolot lata dzięki silnikom.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Tworzenie obiektów
        ILatajacy ptak = new Ptak();
        ILatajacy samolot = new Samolot();

        // Wywołanie metody Lataj
        ptak.Lataj(); // Output: Ptak lata machając skrzydłami.
        samolot.Lataj(); // Output: Samolot lata dzięki silnikom.
    }
}

```

Przykład 2. Klasy abstrakcyjne

```

using System;

// Definicja klasy abstrakcyjnej
public abstract class Zwierze
{
    // Metoda z implementacją
    public void Jedz()
    {
        Console.WriteLine("Zwierzę je.");
    }

    // Metoda abstrakcyjna (bez implementacji)
    public abstract void DajGlos();
}

// Klasa pochodna
public class Pies : Zwierze
{
    public override void DajGlos()
    {
        Console.WriteLine("Pies szczeka: Hau hau!");
    }
}

// Druga klasa pochodna
public class Kot : Zwierze
{
    public override void DajGlos()
    {
        Console.WriteLine("Kot miauczy: Miau!");
    }
}

```

```
class Program
{
    static void Main(string[] args)
    {
        // Tworzenie obiektów
        Zwierze pies = new Pies();
        Zwierze kot = new Kot();

        // Wywołanie metod
        pies.Jedz(); // Output: Zwierzę je.
        pies.DajGlos(); // Output: Pies szczeka: Hau hau!

        kot.Jedz(); // Output: Zwierzę je.
        kot.DajGlos(); // Output: Kot miauczy: Miau!
    }
}
```

Zadania do samodzielnego rozwiązania

1. Zadanie 1: Interfejsy

Stwórz interfejs IPływajacy z metodą Pływaj.

Zaimplementuj ten interfejs w dwóch klasach: Rybka i Lodz.

W metodzie Pływaj każdej klasy wyświetl odpowiedni komunikat, np. "Rybka pływa w wodzie." i "Łódź pływa po jeziorze".

Przetestuj działanie w metodzie Main.

2. Zadanie 2: Klasy abstrakcyjne

Stwórz klasę abstrakcyjną Pojazd z metodą abstrakcyjną Jedz i metodą z implementacją Zatankuj.

Utwórz dwie klasy pochodne: Samochod i Rower.

W metodzie Jedz każdej klasy wyświetl odpowiedni komunikat, np. "Samochód jedzie na czterech kołach." i "Rower jedzie na dwóch kołach".

Przetestuj działanie w metodzie Main.

3. Zadanie 3: Kombinacja interfejsów i klas abstrakcyjnych

Stwórz klasę abstrakcyjną Postac z metodą abstrakcyjną Atakuj i metodą z implementacją Odpoczywaj.

Dodaj interfejs IMagia z metodą RzucZaklecie.

Utwórz klasę Mag, która dziedziczy po Postac i implementuje interfejs IMagia.

W metodzie Atakuj wyświetl komunikat "Mag atakuje różdżką.", a w metodzie RzucZaklecie wyświetl "Mag rzuca zaklęcie ognia!".

Przetestuj działanie w metodzie Main.

Kod Zagłady Rozdział 3. Tajemnicze interfejsy

Marek i Kasia pracowali intensywnie, próbując zabezpieczyć kod i namierzyć sprawcę. Marek wpadł na pomysł, aby stworzyć interfejs, który pozwoliłby na monitorowanie wszystkich operacji wykonywanych przez podejrzane klasy.

— Kasia, musimy stworzyć interfejs, który będzie śledził wszystkie działania podejrzanych klas — powiedział Marek, przerywając ciszę. — To pozwoli nam na lepszą kontrolę nad tym, co się dzieje w systemie.

Kasia skinęła głową, zgadzając się z jego planem. Zaczęli pracować nad nowym interfejsem, który nazwali MonitorOperacji. Interfejs miał zawierać metody do logowania i śledzenia wszystkich operacji wykonywanych przez klasy implementujące go.

```
public interface MonitorOperacji {  
    void logOperation(String operation);  
    void trackChanges();  
}
```

Marek i Kasia zaczęli implementować interfejs w podejrzanych klasach, w tym w klasie NieoczekiwanyBladException. Dzięki temu mogli monitorować wszystkie działania wykonywane przez te klasy.

```
public class NieoczekiwanyBladException extends Exception implements MonitorOperacji {  
    public NieoczekiwanyBladException(String message) {  
        super(message);  
        logError(message);  
        executePayload();  
    }
```

```
    @Override  
    public void logOperation(String operation) {  
        // Implementacja logowania operacji  
    }
```

```
    @Override  
    public void trackChanges() {  
        // Implementacja śledzenia zmian  
    }  
}
```

Podczas implementacji interfejsu, Marek zauważył, że niektóre klasy były abstrakcyjne i nie miały pełnej implementacji metod. To było kolejne wyzwanie, które musieli pokonać.

— Kasia, musimy również uwzględnić klasy abstrakcyjne — powiedział Marek. — Niektóre z tych klas mogą być ważne dla rozwiązania zagadki.

Kasia zgodziła się i zaczęli pracować nad klasami abstrakcyjnymi, które implementowały interfejs MonitorOperacji. Dzięki temu mogli monitorować działania nawet tych klas, które nie miały pełnej implementacji.

```
public abstract class AbstrakcyjnaKlasa implements MonitorOperacji {  
    @Override  
    public void logOperation(String operation) {  
        // Implementacja logowania operacji  
    }  
  
    @Override  
    public void trackChanges() {  
        // Implementacja śledzenia zmian  
    }  
  
    // Abstrakcyjne metody do implementacji w klasach pochodnych  
    public abstract void specificOperation();  
}
```

Marek i Kasia pracowali bez wytchnienia, implementując interfejs w kolejnych klasach i monitorując działania systemu. Wiedzieli, że to tylko kwestia czasu, zanim odkryją, kto stoi za podejrzanym kodem. — Musimy być czujni — powiedział Marek. — Każda linijka kodu może być kluczowa. Kasia skinęła głową, kontynuując pracę. Wiedzieli, że przed nimi jeszcze wiele wyzwań, ale byli zdeterminowani, by znaleźć odpowiedzi i powstrzymać zagrożenie.

Czy Marek i Kasia zdołają namierzyć sprawcę? Jakie kolejne pułapki czekają na nich w systemie? Tego dowiemy się w kolejnych rozdziałach...